

Detecção de Fraude em Transações de Cartão de Crédito com Machine Learning

Relatório Técnico do Projeto Final

Universidade Federal do Amapá (UNIFAP)
CC0121 — Inteligência Artificial — 2026.1

Michel Batista do Monte — 2019007467
Thiago Lobato Rodrigues — 2021009558
Welliton Nunes Almeida — 2019010560

Apresentação Final: 09/07/2026

1. Problema e Motivação

Fraudes em transações de cartão de crédito geram prejuízos bilionários por ano para bancos, fintechs e consumidores. O impacto não é só financeiro: cada fraude não detectada significa dinheiro perdido pela instituição (ou pelo cliente, dependendo da política de estorno), e cada transação legítima bloqueada por engano significa um cliente com o cartão recusado no caixa, ou uma compra online cancelada sem motivo aparente — atrito que corrói confiança na instituição.

O volume de transações processadas por segundo em qualquer operadora de cartões torna a revisão manual inviável. Regras fixas do tipo *if/else* (por exemplo, "bloquear compras acima de R\$ 5.000 fora do horário comercial") são fáceis de contornar, geram falsos positivos em massa para qualquer cliente com hábito de consumo fora do padrão, e não se adaptam a novos padrões de fraude à medida que eles surgem. O problema é multidimensional: envolve dezenas de variáveis correlacionadas entre si (valor, horário, padrão comportamental implícito nas features anonimizadas) que um conjunto de regras estáticas não consegue capturar de forma equilibrada. Um modelo de Machine Learning, por outro lado, aprende esses padrões diretamente dos dados históricos e generaliza para casos novos, ajustando o compromisso entre bloquear fraudes e não incomodar clientes legítimos de forma muito mais granular do que qualquer regra manual.

O objetivo deste projeto é construir um classificador automatizado que identifique transações fraudulentas em tempo real, entregando ao usuário (ou ao sistema antifraude de um banco/fintech) não só uma decisão binária, mas também o nível de confiança da predição — informação essencial para decidir entre bloqueio automático e revisão humana.

2. Dados

2.1 Fonte e características

Utilizamos o dataset público **Credit Card Fraud Detection**, disponibilizado no Kaggle (mlg-ulb/creditcardfraud). Ele contém 284.807 transações reais de cartões europeus, registradas ao longo de dois dias em setembro de 2013. Das 284.807 transações, apenas 492 são fraudulentas — 0,17% do total. O dataset traz 30 features numéricas: Time (segundos desde a primeira transação do conjunto), Amount (valor da transação) e 28 componentes (V1 a V28) resultantes de uma transformação PCA aplicada pelos disponibilizadores originais do dataset sobre os dados brutos, por questões de confidencialidade — não temos acesso aos atributos originais (nome do titular, localização, categoria do estabelecimento, etc.), apenas às componentes anonimizadas.

2.2 O que a análise exploratória revelou

O achado mais determinante da EDA foi o desbalanceamento extremo de classes, ilustrado na Figura 1: em escala logarítmica, a diferença entre 284.315 transações legítimas e 492 fraudulentas fica visualmente clara. Esse desbalanceamento condicionou toda a estratégia de modelagem subsequente — um classificador ingênuo que sempre prevê "legítima" já atinge 99,83% de acurácia sem aprender absolutamente nada sobre fraude, o que torna a acurácia uma métrica inútil neste problema (voltamos a esse ponto na Seção 4).

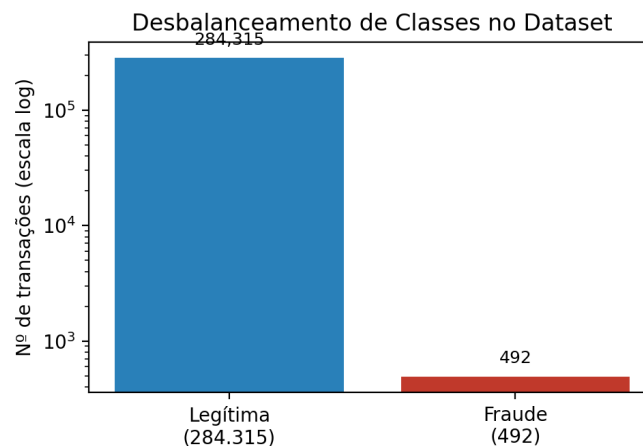


Figura 1 — Desbalanceamento de classes no dataset (escala logarítmica).

Também observamos que a variável Amount tem distribuição fortemente assimétrica (a maioria das transações é de baixo valor, com uma cauda longa de valores altos), e que transações fraudulentas tendem a se concentrar em faixas de valor distintas das legítimas — o que motivou normalizar essa coluna antes da modelagem. Na análise por hora do dia (derivada de Time), fraudes não seguem exatamente o mesmo padrão temporal das transações legítimas, sugerindo que o horário carrega algum poder preditivo residual, ainda que menor que o das componentes PCA mais correlacionadas com a variável alvo.

2.3 Tratamento de problemas nos dados

- **Valores faltantes:** o dataset não apresentou valores nulos em nenhuma coluna (verificado com `isnull().sum()`), então nenhuma imputação foi necessária.
- **Zeros ou valores impossíveis:** não foram encontrados valores de Amount negativos ou inconsistências óbvias nas componentes PCA (que, por definição, podem assumir valores positivos ou negativos livremente).
- **Desbalanceamento de classes:** o problema central dos dados. Tratado com **SMOTE** (Synthetic Minority Over-sampling Technique), que gera exemplos sintéticos da classe minoritária (fraude) por interpolação entre vizinhos próximos no espaço de features — aplicado com balanceamento total (50/50) e, crucialmente, **somente sobre o conjunto de treino**, após o split. Aplicar SMOTE antes do split, ou sobre o conjunto de teste, causaria vazamento de dados (data leakage): exemplos sintéticos derivados de uma transação de teste poderiam acabar no treino, inflando artificialmente o desempenho medido.

3. Metodologia

3.1 Divisão treino/teste

Os dados foram divididos em 80% treino / 20% teste com `train_test_split(..., stratify=y)`, garantindo que a proporção de fraudes (0,17%) se mantivesse praticamente idêntica nos dois conjuntos (0,173% no treino, 0,172% no teste). O split foi feito **antes** de qualquer normalização ou balanceamento, para que nenhuma informação do conjunto de teste influenciasse o pré-processamento do treino.

3.2 Pré-processamento

As componentes V1–V28 já chegam padronizadas pelo PCA original. Apenas Amount e Time foram normalizadas com `StandardScaler`, ajustado (`fit`) exclusivamente sobre o treino e aplicado

(transform) em treino e teste — mesma lógica de evitar vazamento de dados discutida na seção anterior.

3.3 Modelos testados e escolha final

Testamos três algoritmos, todos treinados sobre o conjunto de treino balanceado por SMOTE e avaliados sobre o conjunto de teste original (desbalanceado, refletindo a distribuição real do problema):

- **Regressão Logística** — baseline linear, simples e interpretável.
- **Random Forest** — ensemble de árvores, robusto a features em escalas diferentes e a relações não-lineares.
- **XGBoost** — gradient boosting, referência em desempenho para dados tabulares.

O **Random Forest** foi escolhido como modelo final. Embora o XGBoost tenha alcançado AUC-ROC ligeiramente superior (0,9808 contra 0,9754), ele não atingiu a meta de F1-Score definida na proposta (0,7870 contra o mínimo de 0,80), enquanto o Random Forest atingiu as duas metas simultaneamente (Seção 4 detalha os números). Como o F1 equilibra Precision e Recall — as duas métricas com consequência prática direta neste problema — priorizamos esse critério sobre um ganho marginal de AUC-ROC.

3.4 Como evitamos overfitting

- **Separação estrita treino/teste:** o conjunto de teste nunca foi usado para ajustar hiperparâmetros, escalar dados ou aplicar SMOTE — apenas para avaliação final.
- **SMOTE somente no treino:** evita que exemplos sintéticos "vazem" informação do conjunto de teste para o modelo.
- **Stratified K-Fold Cross-Validation (5 folds):** aplicada sobre o conjunto de treino balanceado do modelo escolhido, mantendo a proporção de classes em cada fold, para confirmar que o desempenho medido é estável e não fruto de um split particularmente favorável.
- **Random Forest com múltiplas árvores (n=200):** o próprio ensemble reduz variância em relação a uma única árvore de decisão, mitigando overfitting a padrões espúrios do treino.

4. Resultados

4.1 Por que F1-Score (e não acurácia)

Com 99,83% dos dados pertencentes à classe "legítima", a acurácia é uma métrica praticamente inútil neste problema: um modelo trivial que nunca prevê fraude atinge 99,83% de acurácia sem qualquer utilidade prática. Por isso, usamos **F1-Score da classe fraude** como métrica principal de decisão — ela equilibra Precision (dos alertas gerados, quantos são fraude de fato) e Recall (das fraudes reais, quantas o modelo captura), as duas métricas com custo de negócio direto: Precision baixa gera excesso de bloqueios em clientes legítimos; Recall baixo deixa fraudes passarem. Complementarmente, usamos AUC-ROC para avaliar a capacidade geral do modelo de separar as duas classes, independente do limiar de decisão escolhido.

4.2 Comparação entre os modelos

Modelo	AUC-ROC	F1 (fraude)	Precision	Recall
Random Forest (escolhido)	0,9754	0,8497	0,8632	0,8367
XGBoost	0,9808	0,7870	0,7203	0,8673
Regressão Logística (baseline)	0,9699	0,1092	0,0580	0,9184

Tabela 1 — Métricas dos três modelos sobre o conjunto de teste ($n=56.962$, sendo 98 fraudes). Meta da proposta: $AUC-ROC \geq 0,95$ e $F1 \text{ (fraude)} \geq 0,80$.

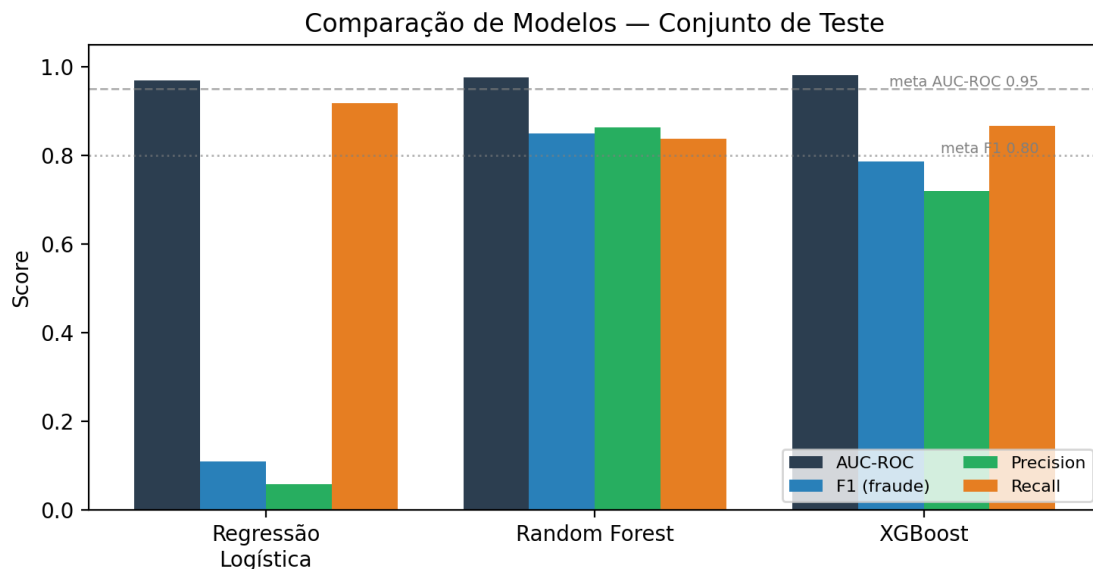


Figura 2 — Comparação visual das quatro métricas entre os três modelos, com as metas da proposta marcadas em cinza.

A Regressão Logística ilustra bem a armadilha de olhar só para o Recall: seus 0,9184 de Recall parecem excelentes à primeira vista, mas vêm de um modelo que classifica quase qualquer transação como fraude (Precision de apenas 0,058) — na prática, um sistema inutilizável, que geraria alertas em massa para transações legítimas.

4.3 Matriz de confusão do modelo escolhido

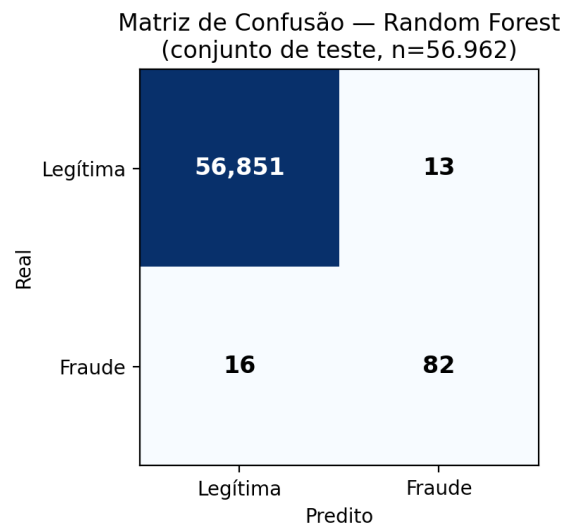


Figura 3 — Matriz de confusão do Random Forest sobre o conjunto de teste (98 fraudes reais em 56.962 transações), reconstruída a partir de Precision e Recall reportados.

Em termos práticos: das 98 fraudes presentes no conjunto de teste, o modelo identificou corretamente 82 (Recall de 83,7%), deixando 16 passarem despercebidas. Dos 95 alertas de fraude gerados no total, 82 eram fraudes reais e 13 eram falsos positivos (Precision de 86,3%) — ou seja, para cada 7 alertas emitidos, aproximadamente 6 são fraudes de fato.

5. Ética e Limitações

5.1 Quem pode ser prejudicado

- **Clientes legítimos com falsos positivos (13 casos no teste):** têm compras recusadas ou cartões bloqueados sem terem cometido fraude. Esse custo não é distribuído igualmente — clientes com padrões de consumo atípicos em relação à maioria dos dados de treino (por exemplo, viajantes frequentes, pequenos empresários com fluxo de caixa irregular, ou pessoas que fazem compras de valores incomuns para seu histórico) tendem a ser desproporcionalmente sinalizados, mesmo sendo legítimos.
- **Vítimas de fraude nos 16 falsos negativos:** fraudes não capturadas resultam em prejuízo financeiro direto ao titular do cartão ou à instituição, dependendo da política de estorno.
- **Grupos sub-representados nos dados:** como o dataset é de apenas dois dias de transações de cartões europeus em 2013, populações, países, moedas e hábitos de consumo fora desse recorte não têm representação alguma no treino — não há garantia de que o modelo generalize de forma justa para outros contextos.

5.2 Viés possível nos dados

As features V1–V28 são anonimizadas via PCA, o que impede qualquer auditoria direta de viés por atributos sensíveis (gênero, renda, localização, etnia) — não sabemos quais combinações lineares dessas variáveis originais compõem cada componente. Isso é positivo sob a ótica de privacidade, mas também é uma limitação: não é possível confirmar nem descartar que o modelo aprenda a discriminar indiretamente por proxies socioeconômicos presentes nos dados originais antes da anonimização (por exemplo, se padrões de valor e horário de transação estiverem correlacionados com perfil de renda). Além disso, o dataset reflete um único processador de pagamentos europeu num recorte temporal muito curto — qualquer viés geográfico, cultural ou temporal presente na coleta original se propaga para o modelo sem que tenhamos como identificá-lo.

5.3 O que o modelo NÃO consegue fazer

- Não explica *por que* classificou uma transação como fraude de forma interpretável para um analista humano ou para o próprio cliente — Random Forest é, na prática, uma caixa-preta sem uma camada de explicabilidade implementada neste projeto (ver Seção 6).
- Não foi validado contra *drift* temporal: os dados são de 2013 e padrões de fraude evoluem constantemente; não há garantia de que o desempenho medido se mantenha sobre transações atuais.
- Não foi testado em ambiente de produção com restrições reais de latência — o pipeline atual (notebook → app Streamlit) é um protótipo de validação, não um sistema de decisão em tempo real sob carga.
- Não incorpora custo de negócio assimétrico entre falso positivo e falso negativo — o limiar de decisão padrão (0,5) não foi ajustado para refletir que, em muitos contextos bancários, o custo de deixar passar uma fraude é diferente do custo de incomodar um cliente legítimo.

6. Conclusão e Próximos Passos

O projeto atingiu as duas metas definidas na proposta ($AUC-ROC \geq 0,95$ e $F1 \geq 0,80$) com o modelo Random Forest, treinado sobre dados balanceados por SMOTE e validado com Stratified K-Fold. O pipeline completo — EDA, pré-processamento, modelagem comparativa e uma interface Streamlit para simulação — demonstra que é possível construir um classificador de fraude funcional mesmo diante do desbalanceamento extremo característico deste tipo de problema.

Com mais um mês de desenvolvimento, priorizaríamos:

- **Explicabilidade (SHAP ou LIME):** entender quais componentes mais influenciam cada predição, permitindo auditar possíveis vieses indiretos e dar suporte a decisões de analistas humanos.
- **Validação temporal:** testar o modelo em dados de períodos distintos (se disponíveis) para estimar sua robustez a drift, em vez de um único split aleatório.
- **Ajuste de limiar por custo de negócio:** em vez do limiar padrão de 0,5, calibrar a fronteira de decisão de acordo com uma matriz de custo real (custo de um falso positivo vs. custo de um falso negativo).
- **Otimização de hiperparâmetros:** busca sistemática (Grid/Random/Bayesian Search) sobre Random Forest e XGBoost, e possivelmente um modelo de stacking combinando os dois.
- **Teste de carga e latência:** avaliar se o pipeline de inferência atende a requisitos de tempo real de um sistema de pagamentos, com benchmarks de throughput.
- **Dados mais representativos:** buscar (ou simular) dados de outros períodos, geografias e processadores, para reduzir o risco de viés discutido na Seção 5.

Referências

- Dal Pozzolo, A. et al. Credit Card Fraud Detection dataset. Kaggle. Disponível em: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.
- Chawla, N. V. et al. SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 2002.
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011.
- Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD, 2016.
- Breiman, L. Random Forests. Machine Learning, 45(1), 5-32, 2001.
- Streamlit Inc. Streamlit Documentation. Disponível em: <https://docs.streamlit.io>.